

字谱 HanziScore

Codex 辅助开发说明 · 轻量版

一句话目标

做一个可以记录汉字书写轨迹、回放书写过程、计算基础指标，并让 AI 用自然语言解释这些指标的小型网页应用。

开发约束

- 五天内完成可演示版本。
- 使用 Flask + 原生 HTML/CSS/JavaScript。
- 不使用 React、数据库、登录、Docker、复杂部署或模型训练。
- AI 接口可选，系统必须在没有网络时仍能展示核心流程。
- 优先保证“能写、能记录、能回放、能解释”，不追求功能完整。

适用方式：把本文档交给 Codex，并按“分阶段提示词”逐步执行。

版本：MVP 0.1 | 用途：AI Practice 组会演示

01 先看这一页：你到底要做什么

这一页既可以给自己看，也可以作为交给 Codex 的项目摘要。

核心问题

传统汉字图像只保存“最后长什么样”。本项目保存“它是怎样被写出来的”，包括笔画顺序、轨迹、时间、速度、停顿，以及设备支持时的压力。

最终演示流程

步骤	模块	用户看到的结果
1	写字	在网页画布上用鼠标或手写笔写一个汉字。
2	记录	系统记录每一笔的坐标、相对时间和可用的压力值。
3	分析	程序计算笔画数、总时长、每笔时长、轨迹长度、平均速度和笔间停顿。
4	回放	按照原始时间顺序重新播放书写过程。
5	AI 解读	AI 把数值转成简洁、有依据的过程描述。

最小成功标准

- 用户能在画布中完成一次书写，抬笔后形成独立 stroke。
- 点击“分析”后，页面能显示至少 5 个客观指标。
- 点击“回放”后，笔迹按原始顺序和大致节奏重新出现。
- 页面能显示一段 AI 或缓存的自然语言解释。
- 即使 AI 接口暂时不可用，其他功能也不能报错或失效。

不要把它做成什么

它不是汉字识别器、笔顺纠错器、书法评分器、字体生成模型，也不是专业绘画软件。第一版只回答：“这个字是怎样被写出来的？”

02 项目范围：只做最小闭环

范围越明确，Codex 越不容易擅自把项目做复杂。

MVP 必做功能

功能	说明	优先级
画布书写	支持 pointerdown / pointermove / pointerup；显示黑色圆头笔迹。	必须
基础操作	清空、撤销上一笔、加载示例。	必须
轨迹记录	记录 x、y、t、pressure；每次落笔到抬笔是一笔。	必须
本地分析	计算笔画数、总时长、每笔时长、轨迹长度、平均速度、笔间停顿。	必须
过程回放	按照原始时间回放，允许停止或重新播放。	必须
结果展示	指标卡片 + 笔画时间轴 + 文本解释。	必须
AI 解读	后端调用模型接口；无密钥时读取本地缓存示例。	建议完成
JSON 导出	下载一次书写的原始数据与分析结果。	有余力再做

明不做

排除项	原因与边界
用户系统	不做登录、注册、权限、账户。
数据存储	不接数据库；刷新页面后数据可丢失。
复杂前端	不使用 React、Vue、TypeScript、Webpack 或组件库。
模型训练	不训练任何神经网络，不收集大规模数据。
识别与评价	不识别写的是哪个字，不判断笔顺和书法水平。
多设备适配	只保证桌面浏览器；手机与平板适配不是本周目标。
线上部署	组会前以本机运行优先，部署不是必要条件。

范围冻结规则

当 Codex 建议新增数据库、框架、用户系统、实时协作、复杂图表库或训练模型时，一律拒绝。只有在核心闭环稳定后，才考虑增加 JSON 导出或两次书写对比。

03 最轻量的技术方案

所有代码都放在一个小型 Flask 项目中。

技术栈

部分	选择	理由
后端	Python + Flask	提供页面、计算特征、可选调用 AI。
前端	原生 HTML + CSS + JavaScript	不需要构建工具，直接由浏览器运行。
绘制	HTML Canvas + Pointer Events	统一处理鼠标、触控和手写笔。
数据	内存中的 JavaScript 对象 / JSON	不使用数据库。
测试	Python 标准库 unittest	只测试核心特征计算，避免额外依赖。

建议目录

目录结构

```
hanziscore/
├── app.py                # Flask 入口与 API
├── analysis.py           # 轨迹特征计算
├── llm_service.py        # AI 接口与缓存回退
├── requirements.txt
├── .env.example
├── AGENTS.md             # 给 Codex 的长期项目规则
├── README.md
├── templates/
│   └── index.html
├── static/
│   ├── style.css
│   └── app.js
├── samples/
│   ├── sample_writing.json
│   └── sample_ai_result.json
└── test_analysis.py
```

页面结构

区域	内容
左侧	600 × 600 书写画布；“撤销”“清空”“回放”“分析”按钮。
右上	笔画数、总时长、最长停顿、平均速度等指标卡片。
右中	简易笔画时间轴，不使用外部图表库。
右下	AI 解读文本，并标明“实时 AI / 缓存示例 / 本地规则”来源。

关键设计

AI 不是项目的唯一支撑。项目先通过普通代码得到可信的数值，再让 AI 负责解释；AI 不可用时，系统仍能展示数据和回放。

04 数据结构：先保证能保存一次书写

所有时间使用相对时间，单位为毫秒。

前端记录的原始数据

writing_data JSON

```
{
  "character": "永",
  "canvas": {"width": 600, "height": 600},
  "device": "pen",
  "strokes": [
    {
      "id": 1,
      "points": [
        {"x": 312.4, "y": 102.8, "t": 0, "pressure": 0.43},
        {"x": 313.1, "y": 108.5, "t": 16, "pressure": 0.47}
      ]
    }
  ]
}
```

记录规则

字段	规则
stroke	一次 pointerdown 到 pointerup / pointercancel 之间的全部点。
x / y	相对于 Canvas 左上角的像素坐标。
t	相对于本次书写第一个点的时间，使用 performance.now() 计算。
pressure	设备支持时记录 0-1；鼠标无真实压力时保存 null。
异常点	一笔少于 2 个点时可以保留，但分析时忽略速度。

后端返回的分析结果

analysis_result JSON

```
{
  "summary": {
    "stroke_count": 5,
    "total_duration_ms": 2310,
    "mean_speed_px_ms": 0.43,
    "longest_pause_ms": 420
  },
  "strokes": [
    {
      "id": 1,
      "duration_ms": 426,
      "path_length_px": 183.2,
      "mean_speed_px_ms": 0.43,
      "pause_before_ms": 0
    }
  ],
  "interpretation": {
    "source": "cached",
    "overall": "本次书写中段节奏较慢，第三笔后停顿最明显。"
  }
}
```

不要伪造压力

鼠标的 pressure 值可能固定或不可用。没有真实手写笔时，页面应显示“当前设备未提供压力数据”，不要把速度模拟的线宽称作真实压力。

05 本地分析：普通程序负责测量

这一部分必须可验证，不能交给大模型猜。

第一版只计算六类指标

指标	计算方式
笔画数	strokes 的数量。
总时长	最后一个有效点 t - 第一个有效点 t。
每笔时长	本笔最后一点 t - 第一点 t。
轨迹长度	相邻坐标点欧氏距离之和。
平均速度	轨迹长度 ÷ 持续时间；持续时间为 0 时返回 null。
笔间停顿	本笔第一点 t - 上一笔最后一点 t。

核心计算伪代码

```
def path_length(points):
    total = 0
    for a, b in consecutive_pairs(points):
        total += sqrt((b.x-a.x)**2 + (b.y-a.y)**2)
    return total

stroke_duration = last_point.t - first_point.t
mean_speed = path_length(points) / stroke_duration
pause_before = current.first.t - previous.last.t
```

必须处理的边界情况

- 没有笔画：返回空结果和友好提示，不抛异常。
- 只有一个点：长度为 0，速度为 null。
- 重复时间戳：避免除以 0。
- pointerup 发生在画布外：通过 pointer capture 或 window 级监听正确结束笔画。
- 回放期间再次书写：先停止回放，再允许输入。
- 用户连续点击“分析”：按钮短暂禁用，避免重复请求。

最低测试用例

测试	预期
两点水平线：(0,0,0) → (3,4,10)	长度 5；时长 10；平均速度 0.5。
两笔间隔：第一笔结束 t=100，第二笔开始 t=180	第二笔 pause_before_ms = 80。
空 strokes	返回 stroke_count=0，不报错。
单点 stroke	长度 0，速度 null。

06 AI 模块：只负责“把数据讲明白”

AI 输出必须有依据，并且可被缓存替代。

AI 的输入与输出

后端只把统计结果发送给模型，不需要发送数百个原始坐标点。这样更轻、更便宜，也更容易控制输出。

输入	输出
总时长、笔画数、各笔持续时间、平均速度、停顿等统计数据。	整体节奏描述。
最快/最慢笔画、最长停顿所在位置。	2-4 条带数据依据的观察。
是否存在压力数据。	不确定性和数据限制。

建议的结构化输出

```
{
  "overall": "整体书写较连续，中段出现一次明显放缓。",
  "observations": [
    {
      "label": "第三笔较慢",
      "evidence": "第三笔平均速度最低，且持续时间最长",
      "confidence": "high"
    }
  ],
  "limitations": ["当前设备没有压力数据"]
}
```


给模型的系统约束

AI system prompt

你是汉字书写事件分析助手。
只根据输入的统计数据描述书写过程。
不要评价书写者的人格、情绪、能力或书法水平。
不要声称识别了标准笔画名称。
每条观察必须给出数据依据。
证据不足时明确写“不确定”。
使用简洁中文，整体不超过 180 字。

三种运行模式

模式	行为	用途
实时 AI	环境变量中有可用密钥，后端真正调用模型。	最理想
缓存 AI	读取 samples/sample_ai_result.json。	现场最稳
本地规则	用 if/else 生成一句简单描述。	最后兜底

诚实标注

页面必须显示当前解释来自“实时 AI”“缓存示例”还是“本地规则”。缓存结果可以保证演示稳定，但不能把缓存冒充成实时调用。

07 如何让 Codex 不把项目做复杂

在仓库根目录放置 AGENTS.md，并让它每次先阅读。

Codex 可以在选定代码目录中读取、修改并运行代码；OpenAI 官方也建议使用 AGENTS.md 保存可复用的仓库规则。具体可执行范围取决于你使用的客户端、权限和批准设置。

建议的 AGENTS.md 内容

AGENTS.md

HanziScore project rules

Goal
Build a five-day lightweight demo for recording and interpreting

```
Chinese-character handwriting events.
```

```
## Required stack
```

- Python + Flask
- Vanilla HTML, CSS and JavaScript
- HTML Canvas and Pointer Events
- JSON files only; no database

```
## Do not add
```

- React, Vue, TypeScript or frontend build tools
- Database, login, Docker or cloud deployment
- Model training or handwriting recognition
- Large UI libraries or chart libraries

```
## Working rules
```

1. Inspect the repository before editing.
2. Make the smallest change that satisfies the current stage.
3. Keep the app runnable after every stage.
4. Run tests after changing analysis.py.
5. Never hard-code API keys.
6. AI failures must fall back to cached or local output.
7. End each task with: changed files, run command, test result, and manual checks for the user.

每次给 Codex 的通用要求

- 先阅读 AGENTS.md、README.md 和现有文件，不要直接重建整个项目。
- 先列出本次将修改的文件和验收标准，再开始编辑。
- 只完成当前阶段，不提前实现后续功能。
- 完成后实际运行程序或测试；不能运行时明确说明原因。
- 报告需要你人工完成的操作，不要假装已经在真实浏览器或手写笔上验证。

为什么要分阶段

一次性让 Codex “完成整个应用” 容易产生过度设计、隐藏错误和难以定位的问题。每完成一个阶段，你都应该亲自运行并截图确认，再进入下一阶段。

08 分阶段提示词（一）：项目骨架与书写记录

下面的文字可以直接复制给 Codex。

阶段 0：只规划，不写代码

复制给 Codex

请先阅读仓库中的 AGENTS.md 和本开发说明。

检查当前目录，不要修改文件。请输出：

1. 最小文件清单；
2. 五个开发阶段；
3. 每阶段验收标准；
4. 你认为仍需我人工决定或提供的内容。

约束：Flask + 原生前端；无数据库；无 React；无部署；

不要建议超出五天 MVP 的功能。

你需要检查

看它是否偷偷加入了数据库、前端框架、复杂依赖或汉字识别。若有，直接要求删除。

阶段 1：创建可运行骨架

复制给 Codex

请创建 HanziScore 的最小可运行项目：

- app.py 提供 GET / 并渲染 templates/index.html；
- static/style.css 和 static/app.js；
- requirements.txt、.env.example、README.md；
- 页面包含标题、600×600 Canvas、撤销/清空/回放/分析按钮，以及空的指标区、时间轴区和 AI 解读区。

暂时不要实现轨迹记录、后端分析或 AI。

完成后运行 Flask，确认首页返回 200，并说明我的启动命令。

你需要做

在自己的电脑上创建并激活 Python 虚拟环境，安装 requirements.txt，打开浏览器确认页面能访问。Codex 不能替你确认你电脑上的端口、防火墙和浏览器显示。

阶段 2：实现书写与数据记录

复制给 Codex

在现有项目上实现 Canvas 书写：

- 使用 Pointer Events；
- pointerdown 开始一笔，pointermove 追加点，pointerup/cancel 结束；
- 记录 x、y、相对 t、pressure（不可用时为 null）；
- 使用 setPointerCapture 或等效方法，避免指针移出画布后丢失抬笔；
- 支持撤销上一笔和清空；
- 画面为白底黑色圆头笔迹；
- 在浏览器控制台提供 getWritingData() 便于检查 JSON。

不要实现后端分析和 AI。完成后说明我应该进行哪些鼠标/手写笔测试。

你需要做

亲自写几个字，并在控制台检查 strokes 数量、点的时间是否递增、撤销是否正确；有数位板时再验证 pressure 是否变化。

09 分阶段提示词（二）：分析、回放与 AI

每一阶段都必须先通过人工验收。

阶段 3：实现本地分析

复制给 Codex

新增 analysis.py，并在 app.py 提供 POST /api/analyze。

输入为 writing_data JSON，返回：

- stroke_count
- total_duration_ms
- 每笔 duration_ms、path_length_px、mean_speed_px_ms
- 每笔 pause_before_ms
- mean_speed_px_ms、longest_pause_ms

要求：

- 空数据、单点笔画、重复时间戳不能报错；
- 非法 JSON 返回清晰的 400 响应；
- 使用 unittest 创建 test_analysis.py，覆盖文档中的四个最低测试；
- 前端“分析”按钮调用接口并显示指标卡片与简易时间轴。

不要加入 AI。完成后运行测试并报告结果。

你需要做

核对至少一个手算样例，确认速度单位和停顿定义符合你的研究意图。Codex 可以计算，但不能替你决定这些指标在研究上是否合理。

阶段 4：实现按原始时间回放

复制给 Codex

在 static/app.js 中实现回放：

- 清空可视画布，但保留 writingData；
- 按点的相对时间逐步重画；
- 笔画间停顿应被保留；
- 回放中禁用书写或先停止回放；

- 清空、重新书写、再次回放不能产生重复计时器；
- 不引入动画库。

请优先保证稳定，不需要倍速、拖动进度条或高级控制。

你需要做

观察回放节奏是否与刚才书写大致一致，并测试连续点击回放、回放中清空、回放后再次书写。真实视觉体验必须由你判断。

阶段 5：接入可回退的 AI 解读

复制给 Codex

新增 llm_service.py，并扩展 /api/analyze：

1. 始终先返回本地统计；
2. 若环境变量 LLM_API_KEY 可用，则调用一个独立的 interpret(metrics) 函数；
3. 若没有密钥、请求失败或超时，则读取 samples/sample_ai_result.json；
4. 缓存也不可用时，用本地规则生成一句话；
5. 返回 interpretation.source = live / cached / local；
6. 不在代码、日志或前端暴露密钥；
7. 模型输出必须被解析为固定 JSON，解析失败时回退。

请把具体模型客户端隔离在 llm_service.py，避免影响其他功能。
README 中说明如何配置，但不要要求我必须使用某一家模型服务。

你需要做

自行选择并准备可用的模型 API、账户、密钥和费用设置；把密钥写入本机 .env，而不是发给 Codex 或提交到仓库。还要审核 AI 文案是否真实、有依据。

10 责任分工：Codex 能做什么，你必须做什么

这是本项目最重要的一张表。

任务	Codex	你必须负责
建立文件和目录	可以	确认项目目录位置，并允许它在正确仓库工作。
编写前后端代码	可以	阅读改动摘要，避免接受不理解的大规模重写。
运行命令和测试	通常可以	根据客户端权限批准命令；在你的实际环境

任务	Codex	你必须负责
		中再次运行。
安装 Python / 修复系统环境	不能完全代替	安装 Python、处理 PATH、端口、权限和本机软件问题。
获得模型 API 密钥	不可以	自行注册服务、确认费用、创建密钥并安全保存。
验证数位板压力	不可以	用真实设备测试 pressure、浏览器兼容性和书写手感。
决定研究定义	不可以	决定“停顿”“速度”“书写事件”等概念如何定义。
判断 AI 解释是否合理	只能辅助	逐条核对数据依据，删除过度推断和书法评价。
准备组会表达	只能辅助	选择演示字、截图、录屏，并亲自排练。
保证现场可用	不可以	准备缓存结果、示例 JSON 和备用演示视频。

你开始前必须亲自完成

1. 在电脑上建立一个全新的 hanziscore 文件夹，并用 VS Code/Codex 打开正确目录。
2. 确认 Python 可以在终端运行，并创建虚拟环境。
3. 决定先使用鼠标完成 MVP，还是同时使用真实手写笔测试压力。
4. 决定是否接入真实模型 API；不接入时先采用缓存输出。
5. 准备一个固定演示字，例如“永”，以及一份备用样例 JSON。

你开发过程中必须亲自完成

1. 每完成一个阶段都打开浏览器测试，不只看 Codex 的文字报告。
2. 检查控制台和终端是否出现错误，遇到问题把完整错误信息发回 Codex。
3. 确认数据定义和单位，而不是默认接受 Codex 的实现。
4. 用真实输入验证撤销、清空、回放、连续点击和异常情况。
5. 审核 AI 输出，确认每个结论都能从数值中得到支持。

11 你需要亲自执行的环境与运行步骤

这些命令可以让 Codex 写进 README，但最终需要你在本机执行。

Windows PowerShell 示例

```
cd 你的项目目录
py -m venv .venv
.\.venv\Scripts\Activate.ps1
pip install -r requirements.txt
python app.py
```

macOS / Linux 示例

```
cd 你的项目目录
python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
python app.py
```

环境变量示例

```
# .env（不要提交）
LLM_API_KEY=你的密钥
LLM_MODEL=你选择的模型
LLM_ENABLED=true

# 没有 API 时
LLM_ENABLED=false
```

最小 requirements.txt

```
Flask
python-dotenv

# 只有接入某个模型服务时，才加入对应官方 SDK。
# 不要为了“以后可能使用”预装大量 AI 框架。
```

密钥安全

不要把真实 API 密钥粘贴进提示词、代码、截图、Git 仓库或组会文档。让 Codex 只读取环境变量名。提交仓库前确认 .env 已写入 .gitignore。

出现问题时怎样给 Codex 信息

错误反馈模板

我运行了：python app.py

操作步骤：写字 → 点击分析
实际结果：页面显示“分析失败”
浏览器控制台错误：<完整复制>
终端错误：<完整复制>
预期结果：应显示指标卡片

请先定位根因，只做最小修复；
不要重写项目，不要新增依赖。

12 五天开发安排与每天验收

每天都留下一版可运行成果，不把所有风险堆到最后。

日期	重点	当天必须通过	留下的成果
Day 1	骨架 + 画布	页面可打开；按钮齐全； Canvas 尺寸正确。	首页截图
Day 2	轨迹记录	能写、撤销、清空；JSON 中 strokes 和 t 正确。	一份 sample_writing.json
Day 3	分析 + 时间轴	接口测试通过；页面显示基础 指标。	测试输出 + 指标截图
Day 4	回放 + AI	回放稳定；实时或缓存解释可 显示并标注来源。	完整 Demo 录屏初版
Day 5	修错 + 汇报	离线也能演示；无明显控制台 错误。	备用视频 + 文档截图

功能优先级下降顺序

1. 首先保证书写记录不丢失。
2. 其次保证本地分析正确。
3. 然后保证回放能工作。
4. 再接入 AI 或缓存解释。
5. 最后才做美化、导出和比较功能。

删功能原则

时间不足时，先删 JSON 导出、速度曲线、两次书写比较和精细动画；不要删轨迹记录、本地指标、回放和解释来源标记。

最终验收清单

- ☐ 启动命令在新终端中可复现。
- ☐ 写一个字后 strokes 数量正确。
- ☐ 撤销只删除最后一笔，清空删除全部数据。
- ☐ 分析空画布时给出友好提示。
- ☐ 回放不会无限叠加计时器。
- ☐ AI 失败不会导致整个页面崩溃。
- ☐ 页面标明解释来源。
- ☐ 没有真实压力时明确显示缺失。
- ☐ .env 不在仓库中。
- ☐ 有一份固定示例 JSON 和一段备用录屏。

13 组会演示脚本与备用方案

控制在 3-5 分钟，先展示理念，再展示技术。

推荐演示脚本

时间	环节	说什么
0:00-0:30	问题	“数字字体通常保存最终图像，但丢失了字是怎样写出来的。”
0:30-1:20	现场书写	写一个“永”，让大家看到系统记录笔画。
1:20-2:10	回放	播放书写过程，说明顺序、速度和停顿被保留下来。
2:10-3:10	指标与 AI	展示客观指标，再展示 AI 如何把数值转成解释。
3:10-4:00	边界	说明当前不做识别、纠错和书法评分。
4:00-5:00	未来	从小工具扩展到汉字书写事件数据集和生成研究。

必须准备的备用材料

- 一份可加载的 sample_writing.json。
- 一份 sample_ai_result.json。
- 一段 30-60 秒的完整操作录屏。

- 3 张关键截图：书写画布、回放过程、最终分析。
- 关闭网络后进行一次完整演练。

遇到现场故障时

故障	处理
AI 接口失败	切换到缓存结果，说明系统采用降级设计。
无法现场写字	加载 sample_writing.json 并演示回放。
浏览器显示异常	播放录屏，同时展示数据结构与理念。
压力没有变化	说明当前设备不提供真实压力，本次只分析轨迹与时间。

本次 AI Practice 的亮点

不是“大模型替我写了一个网页”，而是：你把一个长期研究问题压缩成可运行原型，并让 AI 同时参与开发过程和最终系统中的数据解释。

14 可以直接发送给 Codex 的第一次完整指令

建议先把本 PDF 放在项目目录或将关键内容复制进去。

第一次指令

我要开发一个名为 HanziScore（字谱）的五天轻量 Demo。

它用于记录一个汉字是怎样被写出来的，而不是识别或评价汉字。

最终闭环：

用户在 Canvas 写字 → 记录 strokes/points(x,y,t,pressure) → Flask 计算笔画数、时长、轨迹长度、速度和停顿 → 前端回放 → AI 或缓存结果解释数据。

硬性技术约束：

- Python + Flask
- 原生 HTML/CSS/JavaScript
- Canvas + Pointer Events
- JSON 文件，不使用数据库
- 不使用 React/Vue/TypeScript/构建工具
- 不使用 Docker，不做登录，不做云部署
- 不训练模型，不做汉字识别、笔顺纠错或书法评分
- AI 必须可回退到缓存或本地规则

请先完成以下事情，不要立刻写全部代码：

1. 阅读 AGENTS.md；
2. 检查当前仓库；

- 3. 给出最小目录与五阶段计划；
- 4. 列出每阶段验收标准；
- 5. 列出必须由我亲自完成的环境、设备、API 和语义决策；
- 6. 明确你不会引入哪些额外技术。

等我确认后，只执行阶段 1。每个阶段结束时请报告：

- 修改文件；
- 运行命令；
- 测试结果；
- 我需要亲自检查的项目。

你收到 Codex 计划后只检查三件事

- 1. 有没有超出 MVP：数据库、框架、登录、部署、识别、评分都应删除。
- 2. 有没有明确每个阶段的可运行结果和测试方式。
- 3. 有没有把 API、真实设备、研究定义和人工验收留给你处理。

最重要的协作习惯

不要让 Codex 一次性做完整项目。一次只做一个阶段；你实际运行、测试、截图之后，再把具体问题和错误反馈给它。

完成后可选的唯一扩展

若所有核心功能已经稳定，而且仍有时间，可以增加“两次书写比较”：保存 A 与 B 两次指标，展示总时长和停顿变化，并让 AI 生成一段对比说明。除此之外不再扩展。

附录 Codex 使用参考与人工决策记录

用于确认工具边界，不是项目必须阅读的研究文献。

官方参考

- 1. OpenAI Codex 开发者文档：[Codex overview](#)。说明 Codex 是面向软件开发的编码代理。
- 2. Codex CLI 文档：[Codex CLI](#)。说明本地 CLI 可在选定目录读取、修改并运行代码。
- 3. Codex 最佳实践：[Best practices / AGENTS.md](#)。介绍用 AGENTS.md 保存仓库级长期指令。

开始开发前请填写

需要决定的事项	你的选择
项目保存路径	

需要决定的事项	你的选择
Python 环境 / 版本	_____
主要输入设备：鼠标 / 手写笔	_____
演示汉字	_____
是否接入实时模型 API	是 / 否；服务： _____
“停顿”的定义	_____
AI 输出允许使用的过程标签	_____
组会日期与备用录屏位置	_____

最终原则

Codex 负责加速实现，你负责定义问题、控制范围、验证真实设备、审核 AI 解释，并对组会中展示的结论负责。